The main emergent abilities of LLMs include:

Language ability: LLMs can generate fluent and coherent language.

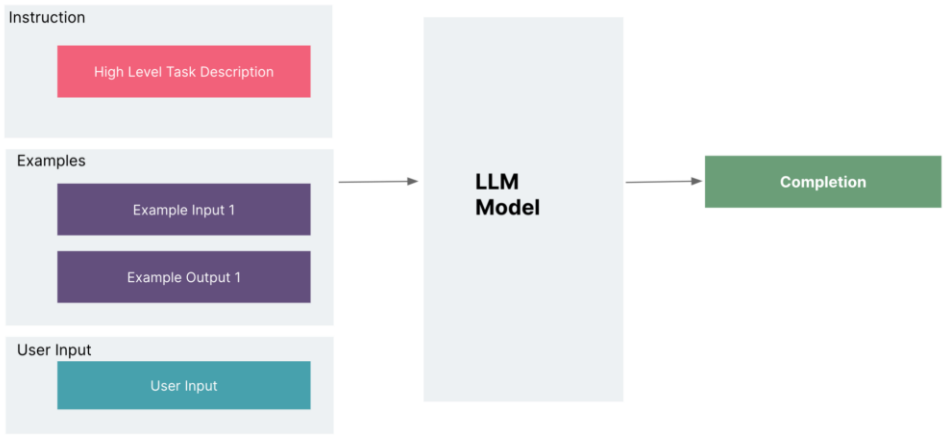
World knowledge: LLMs "remember" vast amounts of knowledge from their training text.

Contextual learning ability: LLMs can learn from input text and extract knowledge without requiring fine-tuning or updating parameters.

Logical reasoning ability: LLMs demonstrate logical reasoning ability, although their reasoning isn't always correct.

Alignment ability: Their ability to "understand" and follow human-language instructions.

These abilities can be useful for a lot more than idle chat. Prompt engineering is one of the ways in which we can fully leverage the abilities of LLMs.



The prompt engineering-centered paradigm has three phases:

Collect a small amount of data to construct an evaluation dataset.

Build prompts based on LLMs and evaluate their effectiveness using the evaluation dataset.

Deploy the prompts to production and track user input and feedback.

Prompt engineering still requires user feedback and prompt iteration. However, its advantages are that it only needs to construct a small evaluation dataset, which greatly reduces the demand for human intervention. In addition, due to LLMs' powerful generalization abilities, prompt engineering can ensure good performance from a cold start. This gives engineers more confidence in estimating the expected

results. Finally, the development and iteration of intelligent products based on prompt engineering is faster than with machine learning models.

Prompt engineering is a relatively new discipline for developing and optimizing prompts to efficiently use LMs for many applications and research topics.

There are different options for building a prompt. But most prompts include some common parts:

Input — This is the actual data the LLM will analyze, like a sentence to translate or document to summarize.

Examples — One good way to show the LLM what to do is by giving a few specific examples of inputs and their correct outputs in the prompt.

Instructions — Instead of examples, the prompt could just clearly tell the LLM what to do in words.

Markers — It helps if the prompt has a predictable structure, so we might separate parts using markers to make it clear.

Context — In addition to the above, sometimes we want to provide more general “context” or background information to the LLM in the prompt as well.

The key parts are usually the input data, examples or instructions, and optionally markers or context. But prompts are flexible — there are different ways to construct one.

Keep it simple: Start with a basic prompt, then slowly make small changes and check the results.

Be direct: If you want the LLM to do something specific like use a certain format, say it clearly up front. Explicitly stating your goal gets the message across best.

Be specific: Vagueness hurts prompts. Give details but don’t overload the prompt with too much information. There’s a limit to how long it can be.

Examples are strong: If it’s hard to describe what you want, concrete samples of correct outputs for different inputs can help show the LLM what to do.

The details vary based on the task and model, but in general it’s good to start basic, be straightforward about your goal, provide details without being vague or adding too much, and examples can reinforce the message if words alone don’t work. Focus on simplicity, clarity and specificity.

Zero-Shot Learning

The concept behind zero-shot learning is quite straightforward. We provide the LLM with a description of the task to complete along with the relevant input data, then have it generate an output. Because LLMs are pretrained on such huge amounts of text, they are often able to solve tasks this way without any fine-tuning or examples. Essentially, they can tap into what they already know to accomplish a variety of jobs. As the examples below show, even something as small as GPT-3.5 could zero-shot learning for different problems simply by being given a prompt describing the task.

Few-Shot Learning

Instead of solely relying on a task description, we can improve our prompt by including clear examples of real inputs and their ideal outputs. This idea underpins few-shot learning, which aims to boost how well an LLM does a job by showing it concrete samples of accurate responses. When done right and paired with a powerful model, few-shot learning is extremely helpful, as evidenced by how effective language models such as GPT-3 became; as shown below, even just a few examples allowed them to solve complex problems. By demonstrating the correct behavior, few-shot learning regularly achieves better results than descriptions alone and was a major factor in recent advancements in large language model capabilities.

The order of examples matters — changing the sequence can greatly change performance. Adding more examples doesn't solve this.

The distribution of labels in examples should match real data. Surprisingly, the labels don't need to be perfect.

LLMs tend to repeat the last example due to recency bias.

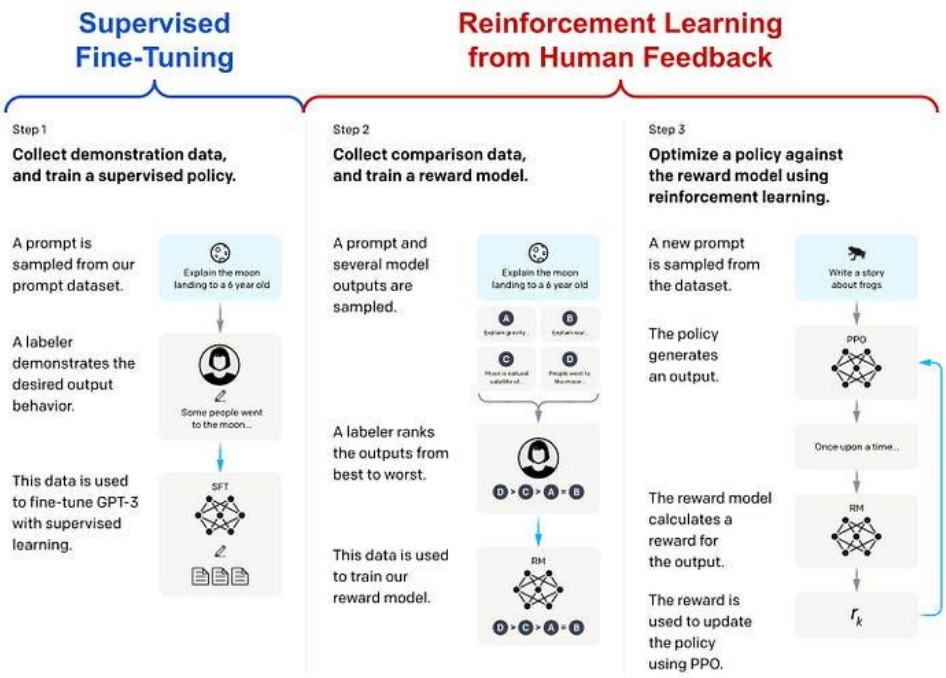
Examples in the prompt should be varied and in random order.

Carefully selecting diverse examples and avoiding biases like repeating the final one can improve few-shot learning results. Prompt construction is critical.

few-shot learning vs. fine-tuning. Before moving forward, I want to clear up a common misunderstanding. Few-shot learning is different than fine-tuning. With few-shot learning, examples are included right in the prompt for context, but the model itself is not changed at all. This is called "in-context learning." Only the prompt is modified, not the model's internal weights. Fine-tuning is when a model is explicitly retrained on a specific dataset, which does alter its parameters through backpropagation. Few-shot learning provides additional information through the prompt instead of modifying the model like fine-tuning does. The key distinction is that few-shot learning does not change the actual model.

Instruction Prompting

While few-shot learning is very powerful, it has a downside — examples use up a lot of the limited context window in an LLM. So we may want methods that are less token-heavy. Is it possible to just explain the right behavior verbally instead of showing samples? The short answer is yes! Giving written directions in the prompt, called instruction prompting, works well — and it's especially effective with a certain type of LLM model.



Here are some tips for writing effective instructions when prompting an instruction-following LLM:

Be very specific and detailed in the instruction.

Avoid saying what not to do — instead focus on clearly stating what you want the LLM to accomplish.

Using markers or indicators in the prompt structure helps distinguish the instruction from other content like examples or input data. This makes the instruction clearer.

The general idea is to precisely yet positively convey what task or outcome you need from the LLM. Being direct and labeling the instruction portion properly helps the model understand and carry out its directions successfully. With an LLM trained for instruction following, taking care with your guidance enables it to help in useful ways.

How to implement a RAG?

Imagine that you want to implement an internal chatbot in your business, to leverage all your company documentation and help to solve questions which may arise. You have documentation stored in a database, so the pieces of knowledge for the Retrieval-augmented Generation will come from there.

The most common approach is to use a specific type of database called a Vector Store, where all your documentation is stored, and an LLM to chat over the documents. This has been “traditionally” carried out in three steps:

Embeddings Calculation. Your documents are processed using a LLM to calculate the Embeddings, a numerical representation of them, and stored in a Vector Store. The Vector Store will act as a database to store the embeddings during indexing time, and retrieve them during answer generation time.

Knowledge Retrieval. During answer generation time, a question triggers a similarity search in the Vector Store, which retrieves the most similar top-X results stored in your Vector Store;

Knowledge Integration. Finally, an LLM is used to combine those responses and write an answer which matches the question asked.

Hallucinations

Hallucinations in LLMs are generally a result of certain underlying factors or causes. While LLMs have been designed to generate coherent and contextually appropriate text, they can occasionally produce hallucinatory or nonsensical responses.

Here are some potential causes of hallucinations in LLMs:

1. Lack of Knowledge. The LLM did not see enough information about the question during training time. In traditional approaches we used to get more data and retrain. But with LLMs that is not always an option due to prohibitive costs of retraining. Other non-prompt related problems are training data bias / sub or over-optimization, noise, etc.

2. Lack of Context. LLMs rely heavily on the prompts to generate responses. If the prompt is insufficient or lacks necessary context, the model may struggle to produce accurate or sensible outputs. Without clear contextual cues, the LLM may fill in the gaps with fabricated or hallucinatory information.

3. Ambiguity. Ambiguous or poorly constructed prompts can lead to misinterpretation by LLMs, resulting in hallucinatory outputs. The model may attempt to make sense of the ambiguous input by generating imaginative or nonsensical responses that are not aligned with the user's intentions.

Tackling point 1 is out of the reach for most of the companies using API-based NLP (e.g. OpenAI's GPT services), since they don't have access to the model. So the only way to properly prevent hallucinations in an LLM consumed via API is tackling points 2 and 3, and this is achieved using Prompt Engineering (PE) techniques.

Now that we covered the main concepts, let's deep dive into what different papers propose in terms of APE building workflows, in the second part of the blog.

Metrics such as BLEU (Bilingual Evaluation Understudy) and ROUGE (Recall-Oriented Understudy for Gisting Evaluation). They are primarily designed to evaluate the quality of machine-generated text, such as translations or summarizations. While they can provide some insights into the quality of an answer to a question, they are not specifically tailored for evaluating question-answering systems. Nevertheless, they can still be used as reference metrics for a rough evaluation. Here's how they work:

BLEU (Bilingual Evaluation Understudy): BLEU is commonly used for evaluating the quality of machine translation output by comparing it to one or more human reference translations. It calculates a score based on n-gram precision, where n-grams are sequences of n words. BLEU evaluates the overlap of n-grams between the generated text and the reference(s), rewarding exact matches and partial matches. A higher BLEU score indicates a closer resemblance to the references.

Applying BLEU to evaluate an answer to a question involves considering the generated answer as a translation of the question. You would need to treat the question as the reference and compare it to the

generated answer using BLEU. However, this approach may not capture the specific nuances and adequacy of the answer to the question.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation): ROUGE is commonly used for evaluating the quality of summarization systems by comparing the generated summary to one or more human reference summaries. It calculates recall scores based on overlapping units, such as n-grams, word sequences, and skip-bigrams. ROUGE assesses the coverage of important information in the generated summary compared to the references.

To evaluate an answer using ROUGE, you would need to treat the generated answer as a summary of the question and compare it to human reference summaries using ROUGE scores. However, similar to BLEU, ROUGE may not fully capture the relevance and accuracy of the answer to the question.

Prompt: A prompt is the input or query provided to an LLM to generate a response. It can be in the form of a sentence, a question, or a more complex instruction.

Context: Context refers to the information or background provided to the model along with the prompt. This can include additional sentences or paragraphs that help the model understand the context of the task.

Prefix: A prefix is a set of initial words or instructions added to a prompt to guide the model's behavior. It can be used to set the context or specify the format of the response.

Length Control: Length control involves specifying how long the generated output should be. This is often done using instructions like "generate a 100-word essay on..." or by setting explicit length limits.

Temperature: Temperature is a hyperparameter that controls the randomness of the model's output. A higher temperature results in more randomness, while a lower temperature makes the output more deterministic.

Top-k Sampling: Top-k sampling is a technique that limits the model's choices during generation by selecting from the top-k most likely words at each step. This can be used to make the output more focused.

Top-p (Nucleus) Sampling: Top-p sampling, also known as nucleus sampling, selects words from the most probable until the cumulative probability exceeds a certain threshold (p). This allows for dynamic control over the level of randomness in the output.

Token: A token is a unit of text in a language model. It can be as short as one character or as long as one word in English.

Fine-tuning: Fine-tuning is the process of training a pre-trained LLM on a specific task or dataset to make it more specialized. It can improve the model's performance for specific applications.

Zero-shot Learning: Zero-shot learning involves using an LLM to perform a task it wasn't explicitly trained for. It relies on the model's general language understanding to generate responses.

Few-shot Learning: Few-shot learning is similar to zero-shot learning but involves providing the model with a few examples or demonstrations of the task before asking it to generate a response.

Demonstration: A demonstration is an example provided to the model to show it how a task should be performed. It is often used in few-shot or guided prompt approaches.

System Output: This refers to the final response generated by the LLM based on the given prompt and instructions.

Rule-based Prompts: These prompts provide explicit rules or instructions to guide the model's behavior, such as "Translate the following English text into French."

Prompt Variations: Experimenting with different prompt phrasings, instructions, or combinations to achieve the desired output is a common practice in prompt engineering.

Bias Mitigation: Prompt engineering can also involve strategies to mitigate biases in LLM outputs, such as using debiasing prompts or instructions.

Evaluation Metrics: Metrics like BLEU, ROUGE, or human evaluation are used to assess the quality and relevance of LLM-generated responses.

https://promptengineering.org/evaluating-the-suitability-of-language-learning-models-for-business-tasks-a-practical-approach/